AI Prompting Lab Reflection Report

Name: Hamza Ali Khan **Course:** AI-101
Submission: AI Prompting Lab 13 Concepts

What I Learned from the Lab

The core lesson running through all 13 exercises is deceptively simple: AI only knows what you tell it. A bare question like "which phone should I buy?" produces advice useful to no one in particular, while the same question loaded with budget, use case, and constraints produces something actually actionable. That shift from generic to specific is what separates a novice prompt from a power-user prompt.

Beyond context, the lab exposed several non-obvious behaviors of AI systems. The "Knows vs. Guesses" exercise highlighted that a confident tone is not a reliability signal. AI can state an incorrect or outdated legal or factual claim with the same fluency it uses to explain photosynthesis. This taught me to always ask: "How would the model even know this?" For anything recent, local, or private, I now either provide the information myself or explicitly ask the model to search and cite sources.

The retrieval modes exercise made me aware that my wording not a button I click determines whether the model answers from its training data, triggers a web search, or structures a deep research report. Words like "today," "latest," and "cite sources" shift the behavior meaningfully.

The data analysis exercise was particularly eye-opening. When asked to calculate statistics without being told to write code, the model can produce plausible-sounding but incorrect numbers through pattern matching alone. Explicitly saying "write and run code, show me the code" is the only reliable way to verify that computation actually happened.

How My Prompting Improved During the Process

At the start, my default approach was to ask a single, short question and then rephrase it when the answer wasn't useful essentially doing multiple rounds of correction that could have been avoided with one well-structured prompt upfront.

By Exercise 4 (Context Is Everything), I adopted the structured briefing format:

I need help with [task]. Here is what you need to know: [constraints, audience, facts the AI couldn't guess]. What I want back: [specific output format].

This alone reduced the number of follow-up corrections significantly. The model stopped filling in assumptions with generic defaults because I had already filled them in correctly.

By Exercise 7 (Brainstorm–Iterate Loop), I stopped expecting the first response to be the final one. Instead I started treating the first answer as a draft to react to picking what worked, naming what didn't, and asking for a revised set. The output after two or three rounds was consistently stronger than anything in the first response.

The sycophancy exercise (Exercise 6) changed how I phrase evaluative questions. Previously I would ask things like "isn't X a better approach?" which signals the expected answer. I now use neutral verbs compare, evaluate, list both sides and explicitly add "don't tell me which is better" when I want the model to present options rather than decide for me.

Which Techniques Helped the Most

1. Structured context upfront (Exercise 4) Providing task, constraints, audience, and desired output format in one prompt eliminated most of the back-and-forth that used to happen in the first two or three messages of any conversation.

2. Brainstorm–Iterate Loop (Exercise 7) Asking for five options instead of one answer, then giving specific feedback before picking a winner, produced noticeably better final results. The value is not in the first answer but in the loop.

3. "Think hard" for multi-trade-off decisions (Exercise 5) On complex decisions involving competing priorities, explicitly invoking deeper reasoning and asking for structured output (trade-offs, recommendation, conditions under which the answer flips) produced more analytical responses than open-ended questions.

4. Neutral framing to avoid sycophancy (Exercise 6) Switching from leading questions to balanced comparison prompts gave me genuinely two-sided analysis instead of confirmation of whatever I implied.

5. Forcing code execution in data tasks (Exercise 10) This was the most practically important technique for anyone doing any quantitative work. Without it, results look correct but may not be.

Challenges Faced and How I Solved Them

Challenge 1: Over-relying on the first answer Early on I accepted the first response too readily, especially when it sounded confident and well-formatted. The "Knows vs. Guesses" and "Data Analysis" exercises made clear that fluency and accuracy are independent. I solved this by building a habit of asking "how would the model know this?" before trusting any factual or numerical claim.

Challenge 2: Prompts that were too long but not actually informative I initially assumed more words meant better context. In practice, adding irrelevant background (things the model didn't need to know) made responses more diffuse. The fix was to focus context on constraints, audience, and output format the three things that most directly shape the answer and cut everything else.

Challenge 3: Getting balanced analysis instead of agreement When I phrased questions with an implied preference, the model tended to agree and justify that preference. Recognizing the sycophancy pattern and switching to neutral comparison verbs consistently fixed this.

Challenge 4: Knowing when a web search had actually run It wasn't always obvious whether the model was answering from training data or from a live search. The solution from Exercise 3 using explicit trigger words like "use a current source and cite it" made the search behavior visible and verifiable through the citations that appeared in the response.

Conclusion

The single principle behind all 13 exercises *get the right context in, keep the wrong context out* turns out to be both the simplest and the hardest thing to do consistently. The lab made that principle concrete through exercises that showed exactly what breaks when context is missing (Exercise 1), when the model doesn't know something but sounds like it does (Exercise 2), and when framing pushes the model toward agreement instead of analysis (Exercise 6). The techniques I'll carry forward most are structured briefing, the brainstorm–iterate loop, neutral comparative framing, and always demanding visible code for any numerical task.
